Tempo di esecuzione

Indice

Tempo di esecuzione di un algoritmo

Ricerca del massimo

Insertion sort

Cambio di base

Test di primalità

Riepilogo

Dove eravamo

- Nel capitolo «Dimensione dell'input» abbiamo imparato a misurare la dimensione $|I|$ di un'istanza di input I
- $|I|$ è il **numero di bit** necessari per rappresentare I
- In questo capitolo usiamo tale misura per analizzare gli algoritmi

La domanda centrale

La domanda centrale dell'analisi degli algoritmi:

come cresce il tempo richiesto da un algoritmo quando cresce l'input?

Piano del capitolo

- Definiamo il tempo di esecuzione di un algoritmo, le operazioni elementari (**passi**) che esso esegue e il **caso peggiore**; fissiamo le convenzioni con cui si contano, riga per riga, i passi di uno pseudocodice
- Applichiamo il metodo a quattro algoritmi già incontrati nel corso: `FindMaximum`, `InsertionSort`, `ChangeToBase2` e `PrimalityTest`
- Per ciascuno: prima il numero di passi $S(n)$ in funzione di un parametro naturale n, poi l'ordine di crescita in funzione della dimensione dell'input $|I|$
- Quattro comportamenti diversi: lineare, quadratico, lineare in $|I|$ benché logaritmico in n, esponenziale in $|I|$ benché solo $\Theta(\sqrt{n})$ in n; li confrontiamo alla fine

Tempo di esecuzione di un algoritmo

Introduzione: Tempo di esecuzione di un algoritmo

- Definiamo il tempo di esecuzione di un algoritmo e le operazioni elementari che esso esegue
- Fissiamo le regole per contare, riga per riga, i passi di uno pseudocodice
- Definiamo il caso peggiore e i due passi dell'analisi usati in tutto il capitolo

Definizione 1: tempo di esecuzione

*Il **tempo di esecuzione** di un algoritmo è una stima del tempo necessario per eseguirlo, in funzione della **dimensione dell'input** $|I|$.*

- Non ci interessa il tempo esatto in secondi, che dipende dal calcolatore, dal compilatore e da molti altri dettagli
- Ci interessa l'**ordine di crescita** del tempo di esecuzione, espresso con la notazione asintotica O, Ω e Θ (capitolo «Notazione asintotica», richiamata nel capitolo «Dimensione dell'input»)

Da che cosa dipende il tempo di esecuzione

Il tempo di esecuzione di un algoritmo dipende dal numero di
operazioni elementari
che l'algoritmo esegue su un input di una data dimensione.

Operazioni elementari

Definizione 2: operazione elementare

*Un'**operazione elementare** è una singola operazione che può essere eseguita da un calcolatore.*

Le principali operazioni elementari sono le seguenti:

1. operazioni aritmetiche su numeri di dimensione fissata;
2. confronto di due numeri di dimensione fissata;
3. assegnamento di un valore a una variabile;
4. accesso a un elemento di un array tramite indice;
5. operazioni di controllo del flusso (per esempio, la valutazione della condizione di un'istruzione `if`, oppure l'iterazione di un ciclo).

Tempo costante e passi

- Assumiamo che per eseguire ciascuna operazione elementare di un algoritmo sia richiesta una **quantità di tempo costante**
- In questo modo l'analisi risulta il più possibile indipendente dalla macchina
- Per brevità, chiamiamo **passo** ogni operazione elementare

Contare i passi di uno pseudocodice

- Ogni riga dei nostri pseudocodici esegue un numero **costante** di operazioni elementari (qualche confronto, operazione aritmetica e assegnamento)
- Poiché ci interessa l'ordine di crescita, contiamo **un passo per ogni esecuzione di una riga**: questo modifica il numero di passi solo per un fattore costante
- Per contare i passi si determina, per ogni riga, quante volte viene eseguita, e si sommano questi numeri

Le convenzioni seguenti sono usate in tutto il capitolo (e nei suoi esercizi).

Conteggio riga per riga (1/2)

- Ogni esecuzione di una riga numerata è **un passo**, comprese le righe `return`
- La condizione di un ciclo `while` il cui corpo viene eseguito k volte viene valutata $k + 1$ volte: l'ultima valutazione è quella che trova la condizione falsa e termina il ciclo
- La condizione di un `if` viene valutata ogni volta che si raggiunge la riga; le righe del ramo che non viene percorso non vengono eseguite

Conteggio riga per riga (2/2)

- Quando il numero di esecuzioni di una riga dipende dallo specifico input (e non soltanto dalla sua dimensione), lo descriviamo con una **funzione ausiliaria**, di solito indicata con t
- Poi cerchiamo l'input che rende il numero totale di passi il più grande possibile

Tempo di esecuzione nel caso peggiore

- Anche per input di una data dimensione, il tempo di esecuzione di un algoritmo può dipendere da quale specifico input di quella dimensione viene fornito

- Tipicamente, nell'analisi degli algoritmi si considera il **caso peggiore possibile**, perché esso determina il tempo di esecuzione più lungo per qualunque input di una data dimensione

Definizione 3: tempo di esecuzione nel caso peggiore

*Il **tempo di esecuzione nel caso peggiore** $T(|I|)$ di un algoritmo è il massimo numero di passi eseguiti dall'algoritmo su un qualunque input I di una data dimensione $|I|$.*

Parametro naturale e dimensione dell'input

- Di solito è più facile contare i passi in funzione di un **parametro naturale** dell'input, come
  - il numero n di elementi di un array, oppure
  - il valore n del numero in input
- La dimensione dell'input $|I|$, invece, è il numero di bit dell'input
- Questo porta al metodo seguito in tutti gli esempi del capitolo

I due passi dell'analisi

1. Contare il numero di passi $S(n)$ eseguiti nel caso peggiore, in funzione del parametro naturale n, e determinarne l'ordine di crescita (per esempio $S(n) = \Theta(n^2)$).

2. Determinare la relazione tra n e la dimensione dell'input $|I|$ (per esempio $|I| = \Theta(n)$, oppure $|I| = \Theta(\log_2 n)$), e usarla per esprimere l'ordine di crescita del tempo di esecuzione nel caso peggiore $T(|I|)$.

Il secondo passo non è una formalità

Lo stesso numero di passi $\Theta(n)$
corrisponde a un tempo di esecuzione

lineare in $|I|$ se n è il numero di elementi di un array,

ed **esponenziale** in $|I|$ se n è il valore del numero in input.

Riepilogo

Tempo di esecuzione:

- Una stima del tempo in funzione di $|I|$, misurata dal suo ordine di crescita
- Si misura in **passi**: operazioni elementari, ciascuna eseguita in tempo costante

Conteggio:

- Un passo per ogni esecuzione di una riga; la condizione di un `while` viene valutata una volta in più del suo corpo
- Il **caso peggiore**: il massimo numero di passi sugli input di una data dimensione

Due passi: prima $S(n)$ in un parametro naturale n, poi $T(|I|)$ nella dimensione dell'input

Ricerca del massimo

Introduzione: Ricerca del massimo

- Il nostro primo esempio: il ciclo più semplice su un array
- Contiamo i passi riga per riga e troviamo l'input del caso peggiore
- Poi esprimiamo il tempo di esecuzione in funzione della dimensione dell'input, nel caso generale e con elementi di un numero fissato di bit

Il problema

Dato un array $a[1..n]$ di $n \geq 1$ numeri interi,
consideriamo il problema di determinare
il **valore massimo** nell'array.

- Nella prossima slide, lo pseudocodice di un semplice algoritmo, chiamato `FindMaximum`, che risolve il problema

Algorithm: FindMaximum

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output : Il valore massimo nell'array a

```
1  function FindMaximum(a, n)
2      max := a[1]
3      i := 2
4      while i ≤ n do
5          if a[i] > max then
6              max ← a[i]
7          end
8          i ← i + 1
9      end
10     return max
11 end
```

Esegui in Colab: Python · C++ · Java

Numero di passi

- Sia $S(n)$ il numero di passi eseguiti da `FindMaximum` nel caso peggiore, dove n è la dimensione dell'array
- Il corpo del ciclo viene eseguito per $i = 2, 3, \ldots, n$, cioè $n - 1$ volte, quindi la condizione della riga 4 viene valutata n volte
- L'esecuzione della riga 6, invece, dipende dall'iterazione. Definiamo una funzione

$$t : \{2, 3, \ldots, n\} \to \{0, 1\},$$

dove $t(i) = 1$ se la riga 6 viene eseguita all'iterazione i, e $t(i) = 0$ altrimenti

Numero di esecuzioni di ciascuna riga

riga	numero di esecuzioni
2	1
3	1
4	n
5	$n-1$
6	$\sum_{i=2}^{n} t(i)$
8	$n-1$
10	1

Quindi il numero totale di passi è

$$1+1+n+(n-1)+\sum_{i=2}^{n} t(i)+(n-1)+1 = 3n+1+\sum_{i=2}^{n} t(i).$$

Il caso peggiore

- Il valore di $t(i)$ dipende dallo specifico array in input, quindi analizziamo il caso peggiore possibile
- Il caso peggiore possibile si ha quando gli elementi dell'array sono in **ordine crescente**:

$$a[1] < a[2] < \cdots < a[n]$$

- Allora $t(i) = 1$ per ogni $i \in \{2, 3, \ldots, n\}$: entriamo nell'`if` a ogni iterazione e aggiorniamo ogni volta il valore massimo
- Il numero totale di passi nel caso peggiore è

$$3n + 1 + \sum_{i=2}^{n} 1 = 3n + 1 + (n - 1) = 4n.$$

L'ordine di crescita di $S(n)$

- Quindi,
$$S(n) = 4n = \Theta(n).$$
- Dunque $S(n)$ cresce **linearmente** in n. Infatti,
$$\lim_{n\to\infty} \frac{4n}{n} = 4 \in \mathbb{R}_{>0}.$$

Verifichiamo il conteggio su un piccolo array.

Esempio (1/2)

Consideriamo l'array $a[1..4] = (3, 1, 4, 5)$.

- Iterazione $i = 2$: $a[2] = 1 > 3$ è falsa, quindi $t(2) = 0$
- Iterazione $i = 3$: $a[3] = 4 > 3$ è vera, *max* diventa 4, quindi $t(3) = 1$
- Iterazione $i = 4$: $a[4] = 5 > 4$ è vera, *max* diventa 5, quindi $t(4) = 1$

Esempio (2/2)

- L'algoritmo restituisce 5 dopo

$$3 \cdot 4 + 1 + \big(t(2) + t(3) + t(4)\big) = 13 + 2 = 15$$

passi

- Sull'array crescente $(1, 2, 3, 4)$, invece, $t(2) = t(3) = t(4) = 1$ e l'algoritmo esegue

$$S(4) = 4 \cdot 4 = 16$$

passi, il massimo per $n = 4$

Tempo di esecuzione in funzione di $|I|$

- Determiniamo ora il tempo di esecuzione nel caso peggiore dell'algoritmo, esprimendo l'ordine di crescita del numero di passi in funzione della dimensione dell'input $|I|$
- Due casi:
  - il **caso generale**, senza ipotesi sugli elementi
  - elementi con un **numero fissato di bit**

Caso generale (1/2)

- L'input è costituito da un array $a[1..n]$ di n numeri interi, quindi la dimensione dell'input soddisfa

$$|I| = \Omega(n),$$

perché l'input deve contenere almeno gli n elementi dell'array

- Questo significa che, per una certa costante $c > 0$ e per ogni n sufficientemente grande,

$$|I| \geq c\, n.$$

- Equivalentemente,

$$n \leq \frac{1}{c}|I|,$$

e quindi $n = O(|I|)$

Caso generale (2/2)

- Poiché $S(n) = \Theta(n)$, esiste una costante $a > 0$ tale che

$$S(n) \leq a\, n$$

per ogni n sufficientemente grande

- Poiché $n \leq \frac{1}{c}|I|$, otteniamo

$$S(n) \leq \frac{a}{c}|I|.$$

- Dunque il numero di passi è limitato superiormente da una costante per la dimensione dell'input

FindMaximum: caso generale

Il tempo di esecuzione nel caso peggiore
dell'algoritmo `FindMaximum`,
in funzione della dimensione dell'input $|I|$, è

$$T(|I|) = O(|I|).$$

Dunque, nel caso generale,
il tempo di esecuzione nel caso peggiore
è **al più lineare** nella dimensione dell'input.

Elementi con un numero fissato di bit (1/2)

- Spesso si fa l'ipotesi che tutti i valori dell'input possano essere memorizzati usando un numero costante fissato di bit, per esempio 32 bit
- Sotto questa ipotesi, la dimensione dell'input soddisfa

$$|I| = \Theta(n),$$

perché l'input è costituito da n elementi dell'array e ciascun elemento richiede un numero costante di bit
- Questo significa che esistono costanti $c_1, c_2 > 0$ tali che, per ogni n sufficientemente grande,

$$c_1 n \leq |I| \leq c_2 n.$$

- Equivalentemente,

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|,$$

cioè $n = \Theta(|I|)$

Elementi con un numero fissato di bit (2/2)

- Poiché $S(n) = \Theta(n)$, esistono costanti $a_1, a_2 > 0$ tali che, per ogni n sufficientemente grande,

$$a_1 n \leq S(n) \leq a_2 n.$$

- Usando i limiti su n in funzione di $|I|$, otteniamo

$$\frac{a_1}{c_2}|I| \leq S(n) \leq \frac{a_2}{c_1}|I|,$$

cioè $S(n) = \Theta(|I|)$

FindMaximum: elementi con un numero fissato di bit

Se tutti i valori dell'input usano
un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore
dell'algoritmo `FindMaximum` è

$$T(|I|) = \Theta(|I|).$$

Dunque, sotto questa ipotesi,
l'algoritmo è **lineare** nella dimensione dell'input.

Riepilogo

Numero di passi:

- $3n + 1 + \sum_{i=2}^{n} t(i)$, dove $t(i) = 1$ se il massimo viene aggiornato all'iterazione i
- Caso peggiore: array crescente, $S(n) = 4n = \Theta(n)$

Tempo di esecuzione:

- Caso generale: $|I| = \Omega(n)$, quindi $T(|I|) = O(|I|)$
- Elementi con un numero fissato di bit: $|I| = \Theta(n)$, quindi $T(|I|) = \Theta(|I|)$

Insertion sort

Introduzione: Insertion sort

- In `FindMaximum` il corpo del ciclo esegue un numero costante di passi
- In `InsertionSort` il corpo del ciclo esterno contiene **un altro ciclo**, il cui numero di iterazioni dipende dall'input
- Questo produce un numero **quadratico** di passi

Il problema

Dato un array $a[1..n]$ di n numeri naturali, consideriamo il problema di ordinare l'array in ordine non decrescente:

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

- Nella prossima slide, lo pseudocodice dell'algoritmo `InsertionSort`

Algorithm: InsertionSort (1/2)

Input : Un array $a[1..n]$ di n numeri naturali
Output : L'array a ordinato in ordine non decrescente

```
1 function InsertionSort(a, n)
2 |   i := 2
3 |   while i ≤ n do
4 |   |   key := a[i]
5 |   |   j := i − 1
```

Algorithm: InsertionSort (2/2)

```
function InsertionSort(a, n) (continua)
    while i ≤ n (continua) do
6       while j ≥ 1 and a[j] > key do
7           a[j + 1] ← a[j]
8           j ← j − 1
9       end
10      a[j + 1] ← key
11      i ← i + 1
12  end
13 end
```

Esegui in Colab: Python · C++ · Java

Numero di passi

- Sia $S(n)$ il numero di passi eseguiti da `InsertionSort` nel caso peggiore, dove n è la dimensione dell'array
- L'esecuzione delle righe 7–8 dipende dall'iterazione. Definiamo una funzione

$$t : \{2, 3, \ldots, n\} \to \{0, 1, \ldots, n-1\},$$

  dove $t(i)$ indica il numero di volte in cui il corpo del ciclo interno (righe 7–8) viene eseguito durante l'iterazione i del ciclo esterno
- La condizione del ciclo interno (riga 6) viene quindi valutata $t(i) + 1$ volte durante la stessa iterazione

Numero di esecuzioni di ciascuna riga

riga	numero di esecuzioni
2	1
3	n
4	$n-1$
5	$n-1$
6	$(n-1)+\sum_{i=2}^{n} t(i)$
7	$\sum_{i=2}^{n} t(i)$
8	$\sum_{i=2}^{n} t(i)$
10	$n-1$
11	$n-1$

Numero totale di passi

Quindi il numero totale di passi è

$$1 + n + (n-1) + (n-1) + \left((n-1) + \sum_{i=2}^{n} t(i) \right)$$

$$+ \sum_{i=2}^{n} t(i) + \sum_{i=2}^{n} t(i) + (n-1) + (n-1)$$

$$= 6n - 4 + 3 \sum_{i=2}^{n} t(i).$$

Il caso peggiore (1/2)

- Il valore di $t(i)$ dipende dallo specifico array in input, quindi analizziamo il caso peggiore possibile
- Poiché j parte da $i-1$ e decresce al più fino a 0, si ha sempre $t(i) \leq i-1$
- Il caso peggiore possibile si ha quando gli elementi dell'array sono in **ordine strettamente decrescente**:

$$a[1] > a[2] > \cdots > a[n]$$

- In questo caso, a ogni iterazione esterna i, l'elemento $key = a[i]$ è più piccolo di tutti gli elementi che lo precedono. Quindi il ciclo while interno viene eseguito fino in fondo:

$$t(i) = i-1 \qquad \forall i \in \{2, 3, \ldots, n\}.$$

Il caso peggiore (2/2)

- Perciò,

$$\sum_{i=2}^{n} t(i) = \sum_{i=2}^{n} (i-1) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

- Il numero totale di passi nel caso peggiore è

$$6n - 4 + 3 \cdot \frac{n(n-1)}{2} = \frac{12n - 8 + 3n^2 - 3n}{2} = \frac{3n^2 + 9n - 8}{2}.$$

L'ordine di crescita di $S(n)$

- Quindi,

$$S(n) = \frac{3n^2 + 9n - 8}{2} = \Theta(n^2).$$

- Dunque $S(n)$ cresce **quadraticamente** in n. Infatti,

$$\lim_{n\to\infty} \frac{3n^2 + 9n - 8}{2n^2} = \lim_{n\to\infty} \frac{1}{2}\left(3 + \frac{9}{n} - \frac{8}{n^2}\right) = \frac{3}{2} \in \mathbb{R}_{>0}.$$

Vediamo la funzione t su un array concreto.

Esempio (1/2)

- Consideriamo l'array $a[1..6] = (5, 2, 4, 6, 1, 3)$
- A ogni iterazione i, $t(i)$ è il numero di elementi di $a[1..i-1]$ più grandi di $key = a[i]$, cioè il numero di elementi spostati di una posizione verso destra

i	key	$t(i)$	array dopo l'iterazione
2	2	1	$(2, 5, 4, 6, 1, 3)$
3	4	1	$(2, 4, 5, 6, 1, 3)$
4	6	0	$(2, 4, 5, 6, 1, 3)$
5	1	4	$(1, 2, 4, 5, 6, 3)$
6	3	3	$(1, 2, 3, 4, 5, 6)$

Esempio (2/2)

- Abbiamo $\sum_{i=2}^{6} t(i) = 9$, quindi l'algoritmo esegue

$$6 \cdot 6 - 4 + 3 \cdot 9 = 59$$

passi

- Su un array decrescente di 6 elementi, invece, $\sum_{i=2}^{6} t(i) = 1 + 2 + 3 + 4 + 5 = 15$ e l'algoritmo esegue

$$S(6) = \frac{3 \cdot 36 + 9 \cdot 6 - 8}{2} = 77$$

passi

Tempo di esecuzione in funzione di $|I|$

- Determiniamo ora il tempo di esecuzione nel caso peggiore dell'algoritmo, esprimendo l'ordine di crescita del numero di passi in funzione della dimensione dell'input $|I|$
- Come per `FindMaximum`, due casi:
  - il **caso generale**
  - elementi con un **numero fissato di bit**

Caso generale (1/2)

- L'input è costituito da un array $a[1..n]$ di n numeri naturali, quindi la dimensione dell'input soddisfa

$$|I| = \Omega(n),$$

perché l'input deve contenere almeno gli n elementi dell'array

- Questo significa che, per una certa costante $c > 0$ e per ogni n sufficientemente grande,

$$|I| \geq c\, n.$$

- Equivalentemente,

$$n \leq \frac{1}{c}|I|,$$

e quindi $n = O(|I|)$

Caso generale (2/2)

- Poiché $S(n) = \Theta(n^2)$, esiste una costante $a > 0$ tale che

$$S(n) \leq a\, n^2$$

per ogni n sufficientemente grande

- Poiché $n \leq \frac{1}{c}|I|$, otteniamo

$$S(n) \leq a \left(\frac{1}{c}|I|\right)^2 = \frac{a}{c^2}|I|^2.$$

- Dunque il numero di passi è limitato superiormente da una costante per $|I|^2$

InsertionSort: caso generale

Il tempo di esecuzione nel caso peggiore
dell'algoritmo `InsertionSort`,
in funzione della dimensione dell'input $|I|$, è

$$T(|I|) = O(|I|^2).$$

Dunque, nel caso generale,
il tempo di esecuzione nel caso peggiore
è **al più quadratico** nella dimensione dell'input.

Elementi con un numero fissato di bit (1/2)

- Spesso si fa l'ipotesi che tutti i valori dell'input possano essere memorizzati usando un numero costante fissato di bit, per esempio 32 bit
- Sotto questa ipotesi, la dimensione dell'input soddisfa

$$|I| = \Theta(n),$$

perché l'input è costituito da n elementi dell'array e ciascun elemento richiede un numero costante di bit

- Questo significa che esistono costanti $c_1, c_2 > 0$ tali che, per ogni n sufficientemente grande,

$$c_1 n \leq |I| \leq c_2 n.$$

- Equivalentemente,

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|.$$

Elementi con un numero fissato di bit (2/2)

- Poiché $S(n) = \Theta(n^2)$, esistono costanti $a_1, a_2 > 0$ tali che, per ogni n sufficientemente grande,

$$a_1 n^2 \leq S(n) \leq a_2 n^2.$$

- Usando i limiti su n in funzione di $|I|$, otteniamo

$$a_1 \left(\frac{1}{c_2}|I|\right)^2 \leq S(n) \leq a_2 \left(\frac{1}{c_1}|I|\right)^2,$$

cioè

$$\frac{a_1}{c_2^2}|I|^2 \leq S(n) \leq \frac{a_2}{c_1^2}|I|^2.$$

- Dunque $S(n) = \Theta(|I|^2)$

InsertionSort: elementi con un numero fissato di bit

Se tutti i valori dell'input usano
un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore
dell'algoritmo `InsertionSort` è

$$T(|I|) = \Theta(|I|^2).$$

Dunque, sotto questa ipotesi,
l'algoritmo è **quadratico** nella dimensione dell'input.

Riepilogo

Numero di passi:

- $6n - 4 + 3\sum_{i=2}^{n} t(i)$, dove $t(i)$ conta le iterazioni del ciclo interno
- Caso peggiore: array strettamente decrescente, $t(i) = i - 1$, $S(n) = \frac{3n^2+9n-8}{2} = \Theta(n^2)$

Tempo di esecuzione:

- Caso generale: $T(|I|) = O(|I|^2)$
- Elementi con un numero fissato di bit: $T(|I|) = \Theta(|I|^2)$

Cambio di base

Introduzione: Cambio di base

- Nei primi due esempi l'input è un array e il parametro naturale n è il numero dei suoi elementi
- Consideriamo ora un algoritmo il cui input è un **singolo numero** n: il parametro n è ora un valore
- La relazione tra n e $|I|$ cambia completamente

Il problema

Dato un numero naturale $n \geq 1$ rappresentato in base 10, consideriamo il problema di convertire n in base 2.

- L'algoritmo `ChangeToBase2` è l'algoritmo `ChangeOfBase` del capitolo «Algoritmi numerici», con base $b = 2$
- Le cifre binarie sono memorizzate in un array $d[0..nd-1]$ indicizzato a partire da 0, in modo che $d[k]$ sia la cifra di peso 2^k

Algorithm: ChangeToBase2 (1/2)

Input : Un numero naturale $n \geq 1$ in base 10
Output : Un array d di cifre binarie e il numero di cifre nd

```
1 function ChangeToBase2(n)
2 |   nd := ⌊log_2 n⌋ + 1
3 |   d := nuovo array[0..nd − 1]
4 |   k := 0
```

Algorithm: ChangeToBase2 (2/2)

```
function ChangeToBase2(n) (continua)
5     while n > 0 do
6         q := ⌊n/2⌋
7         r := n mod 2
8         d[k] ← r
9         n ← q
10        k ← k + 1
11    end
12    return d, nd
13 end
```

Esegui in Colab: Python · C++ · Java

Numero di passi (1/2)

- Sia $S(n)$ il numero di passi eseguiti da `ChangeToBase2`, dove n è il valore in input da convertire
- Per ogni valore di n c'è un solo input, quindi **non c'è distinzione** tra il caso peggiore e gli altri casi
- Il numero di cifre necessarie per rappresentare n in base 2 è

$$\lfloor \log_2 n \rfloor + 1.$$

- A ogni iterazione del ciclo while, il valore corrente di n viene sostituito con $\lfloor n/2 \rfloor$. Dopo s iterazioni, il valore corrente è

$$\left\lfloor \frac{n}{2^s} \right\rfloor.$$

Numero di passi (2/2)

- Il ciclo termina quando questo valore diventa 0, cioè quando

$$\left\lfloor \frac{n}{2^s} \right\rfloor = 0.$$

- Questo accade esattamente quando $2^s > n$. Il più piccolo intero s tale che $2^s > n$ è

$$s = \lfloor \log_2 n \rfloor + 1.$$

- Quindi il corpo del ciclo while viene eseguito esattamente $\lfloor \log_2 n \rfloor + 1$ volte (un'iterazione per ogni cifra binaria), e la sua condizione viene valutata $\lfloor \log_2 n \rfloor + 2$ volte

Numero di esecuzioni di ciascuna riga

riga	numero di esecuzioni
2, 3, 4	1 ciascuna
5	$\lfloor \log_2 n \rfloor + 2$
6, 7, 8, 9, 10	$\lfloor \log_2 n \rfloor + 1$ ciascuna
12	1

Quindi il numero totale di passi è

$$1 + 1 + 1 + (\lfloor \log_2 n \rfloor + 2) + 5(\lfloor \log_2 n \rfloor + 1) + 1$$

$$= 6(\lfloor \log_2 n \rfloor + 1) + 5 = 6\lfloor \log_2 n \rfloor + 11.$$

L'ordine di crescita di $S(n)$ (1/2)

- Quindi,
$$S(n) = 6\lfloor \log_2 n \rfloor + 11 = \Theta(\log_2 n).$$
In parole, il numero di passi cresce in modo **logaritmico** nel valore numerico n
- Infatti, per $n > 1$,
$$\log_2 n - 1 < \lfloor \log_2 n \rfloor \leq \log_2 n.$$
- Dividendo per $\log_2 n > 0$, otteniamo
$$1 - \frac{1}{\log_2 n} < \frac{\lfloor \log_2 n \rfloor}{\log_2 n} \leq 1.$$

L'ordine di crescita di $S(n)$ (2/2)

- Poiché sia $1 - \frac{1}{\log_2 n}$ sia 1 convergono a 1, per il teorema del confronto (dei due carabinieri),

$$\lim_{n\to\infty} \frac{\lfloor \log_2 n \rfloor}{\log_2 n} = 1.$$

- Quindi,

$$\lim_{n\to\infty} \frac{6\lfloor \log_2 n \rfloor + 11}{\log_2 n} = \lim_{n\to\infty} \left(6\,\frac{\lfloor \log_2 n \rfloor}{\log_2 n} + \frac{11}{\log_2 n} \right)$$

$$= 6 \cdot 1 + 0 = 6 \in \mathbb{R}_{>0}.$$

Esempio (1/2)

- Eseguiamo `ChangeToBase2` su $n = 13$
- Abbiamo $\lfloor \log_2 13 \rfloor + 1 = 4$, quindi viene creato l'array $d[0..3]$ e il ciclo esegue 4 iterazioni:

iterazione	n	$q = \lfloor n/2 \rfloor$	$r = n \bmod 2$	assegnamento
1	13	6	1	$d[0] = 1$
2	6	3	0	$d[1] = 0$
3	3	1	1	$d[2] = 1$
4	1	0	1	$d[3] = 1$

Esempio (2/2)

- Dopo la quarta iterazione $n = 0$: la condizione della riga 5 viene valutata per la quinta volta, è falsa, e l'algoritmo restituisce le cifre di $13 = (1101)_2$
- Il numero di passi è

$$S(13) = 3 + 5 + 5 \cdot 4 + 1 = 29 = 6 \cdot 3 + 11.$$

Tempo di esecuzione in funzione di $|I|$ (1/3)

- Esprimiamo ora il tempo di esecuzione in funzione della dimensione dell'input $|I|$. Questo è il **delicato cambio di variabile**
- L'input non è il valore astratto n, ma una **rappresentazione** di n. Poiché n è rappresentato in base 10, il numero di cifre necessarie per scrivere n è

$$\lfloor \log_{10} n \rfloor + 1.$$

- Ogni cifra decimale richiede un numero costante di bit. Quindi la dimensione dell'input soddisfa

$$|I| = \Theta(\lfloor \log_{10} n \rfloor + 1) = \Theta(\log_{10} n).$$

Tempo di esecuzione in funzione di $|I|$ (2/3)

- Poiché il cambio di base di un logaritmo lo modifica solo per un fattore costante ($\log_{10} n = \log_2 n / \log_2 10$), otteniamo

$$|I| = \Theta(\log_2 n),$$

  lo stesso ordine di crescita ottenuto nel capitolo «Dimensione dell'input» per un numero scritto in binario
- Questo significa che esistono costanti $c_1, c_2 > 0$ tali che, per ogni n sufficientemente grande,

$$c_1 \log_2 n \leq |I| \leq c_2 \log_2 n.$$

- Equivalentemente,

$$\frac{1}{c_2}|I| \leq \log_2 n \leq \frac{1}{c_1}|I|.$$

Tempo di esecuzione in funzione di $|I|$ (3/3)

- Poiché $S(n) = \Theta(\log_2 n)$, esistono costanti $a_1, a_2 > 0$ tali che, per ogni n sufficientemente grande,

$$a_1 \log_2 n \leq S(n) \leq a_2 \log_2 n.$$

- Usando i limiti su $\log_2 n$ in funzione di $|I|$, otteniamo

$$\frac{a_1}{c_2}|I| \leq S(n) \leq \frac{a_2}{c_1}|I|,$$

cioè $S(n) = \Theta(|I|)$

ChangeToBase2: tempo di esecuzione

Il tempo di esecuzione dell'algoritmo `ChangeToBase2` è

$$T(|I|) = \Theta(|I|).$$

Dunque l'algoritmo è **logaritmico** nel valore numerico n,
ma **lineare** nella dimensione dell'input $|I|$,
perché la dimensione dell'input
è essa stessa proporzionale a $\log_2 n$.

Riepilogo

Numero di passi:

- Un'iterazione per ogni cifra binaria: $\lfloor \log_2 n \rfloor + 1$ iterazioni
- $S(n) = 6\lfloor \log_2 n \rfloor + 11 = \Theta(\log_2 n)$, lo stesso per ogni input di valore n

Tempo di esecuzione:

- L'input è una rappresentazione di n: $|I| = \Theta(\log_2 n)$
- Quindi $T(|I|) = \Theta(|I|)$: logaritmico in n, lineare in $|I|$

Test di primalità

Introduzione: Test di primalità

- L'input dell'ultimo esempio è di nuovo un singolo numero n
- Ma ora il numero di passi cresce come $\sqrt{n}$, molto più rapidamente di $\log_2 n$
- Vedremo che questo rende l'algoritmo **esponenziale** nella dimensione dell'input

Il problema

Dato un numero naturale n,
consideriamo il problema di determinare
se n è un **numero primo**.

- Nella prossima slide, lo pseudocodice dell'algoritmo `PrimalityTest` del capitolo «Algoritmi numerici»

Algorithm: PrimalityTest

Input : Un numero naturale n
Output : `true` se n è primo, `false` altrimenti

```
1  function PrimalityTest(n)
2      if n < 2 then
3          return false
4      end
5      d := 2
6      while d ≤ ⌊√n⌋ do
7          if n mod d = 0 then
8              return false
9          end
10         d ← d + 1
11     end
12     return true
13 end
```

Esegui in Colab: Python · C++ · Java

Numero di passi (1/2)

- Sia $S(n)$ il numero di passi eseguiti da `PrimalityTest` nel caso peggiore, dove n è il valore in input
- Assumiamo $n \geq 2$, perché per $n < 2$ l'algoritmo si ferma dopo 2 passi
- **Il caso peggiore si ha quando n è primo.** In questo caso l'algoritmo non esce mai in anticipo alla riga 8, quindi il ciclo while viene eseguito fino in fondo

Numero di passi (2/2)

- La variabile d assume i valori

$$2, 3, \ldots, \lfloor \sqrt{n} \rfloor,$$

quindi il corpo del ciclo while viene eseguito $\lfloor \sqrt{n} \rfloor - 1$ volte

- La condizione del ciclo while viene valutata ancora una volta, quando $d = \lfloor \sqrt{n} \rfloor + 1$, quindi la riga 6 viene eseguita $\lfloor \sqrt{n} \rfloor$ volte

Numero di esecuzioni di ciascuna riga

riga	numero di esecuzioni (n primo)
2	1
3	0 (perché $n \geq 2$)
5	1
6	$\lfloor \sqrt{n} \rfloor$
7	$\lfloor \sqrt{n} \rfloor - 1$
8	0 (perché n è primo)
10	$\lfloor \sqrt{n} \rfloor - 1$
12	1

Quindi il numero totale di passi nel caso peggiore è

$$1 + 1 + \lfloor \sqrt{n} \rfloor + (\lfloor \sqrt{n} \rfloor - 1) + (\lfloor \sqrt{n} \rfloor - 1) + 1 = 3\lfloor \sqrt{n} \rfloor + 1.$$

È davvero il caso peggiore

- Se $n \geq 2$ non è primo, sia p il suo più piccolo divisore maggiore di 1: allora $p \leq \lfloor \sqrt{n} \rfloor$, e l'algoritmo restituisce `false` alla riga 8 quando $d = p$

- La riga 6 viene eseguita $p - 1$ volte, la riga 7 $p - 1$ volte, la riga 10 $p - 2$ volte, e le righe 2, 5, 8 una volta ciascuna, per un totale di

$$3p - 1 \;\leq\; 3\lfloor \sqrt{n} \rfloor - 1 \;<\; 3\lfloor \sqrt{n} \rfloor + 1$$

passi

L'ordine di crescita di $S(n)$ (1/2)

- Quindi,
$$S(n) = 3\lfloor\sqrt{n}\rfloor + 1 = \Theta(\sqrt{n}).$$
- Dunque $S(n)$ cresce come la **radice quadrata** di n. Infatti,
$$\sqrt{n} - 1 < \lfloor\sqrt{n}\rfloor \leq \sqrt{n}.$$
- Dividendo per $\sqrt{n} > 0$, otteniamo
$$1 - \frac{1}{\sqrt{n}} < \frac{\lfloor\sqrt{n}\rfloor}{\sqrt{n}} \leq 1.$$

L'ordine di crescita di $S(n)$ (2/2)

- Poiché sia $1 - \frac{1}{\sqrt{n}}$ sia 1 convergono a 1, per il teorema del confronto (dei due carabinieri),

$$\lim_{n\to\infty} \frac{\lfloor\sqrt{n}\rfloor}{\sqrt{n}} = 1.$$

- Quindi,

$$\lim_{n\to\infty} \frac{3\lfloor\sqrt{n}\rfloor + 1}{\sqrt{n}} = \lim_{n\to\infty} \left(3\,\frac{\lfloor\sqrt{n}\rfloor}{\sqrt{n}} + \frac{1}{\sqrt{n}}\right)$$

$$= 3 \cdot 1 + 0 = 3 \in \mathbb{R}_{>0}.$$

Esempio (1/2)

- $n = 97$ è primo e $\lfloor \sqrt{97} \rfloor = 9$
- L'algoritmo prova i divisori $d = 2, 3, \ldots, 9$ (8 iterazioni), non trova alcun divisore, e valuta una nona volta la condizione della riga 6 con $d = 10$
- Restituisce `true` dopo

$$S(97) = 3 \cdot 9 + 1 = 28$$

passi

Esempio (2/2)

- $n = 91 = 7 \cdot 13$ non è primo, e il suo più piccolo divisore maggiore di 1 è $p = 7$
- L'algoritmo prova $d = 2, 3, \ldots, 6$ senza successo, e trova $91 \bmod 7 = 0$
- Restituisce `false` dopo

$$3 \cdot 7 - 1 = 20$$

passi, anche se $\lfloor \sqrt{91} \rfloor = 9$ come per 97

Tempo di esecuzione in funzione di $|I|$ (1/4)

- Determiniamo ora il tempo di esecuzione nel caso peggiore, esprimendo l'ordine di crescita del numero di passi in funzione della dimensione dell'input $|I|$
- L'input è un singolo numero naturale n. Per rappresentare n in binario servono $\lfloor \log_2 n \rfloor + 1$ bit, quindi la dimensione dell'input soddisfa
$$|I| = \Theta(\log_2 n).$$
- Questo significa che esistono costanti $c_1, c_2 > 0$ tali che, per ogni n sufficientemente grande,
$$c_1 \log_2 n \leq |I| \leq c_2 \log_2 n.$$
- Equivalentemente,
$$\frac{1}{c_2}|I| \leq \log_2 n \leq \frac{1}{c_1}|I|.$$

Tempo di esecuzione in funzione di $|I|$ (2/4)

- La funzione $x \mapsto 2^x$ è strettamente crescente. Quindi, applicare 2^x a tutti i termini della disuguaglianza ne preserva l'ordine:
$$2^{\frac{1}{c_2}|I|} \leq 2^{\log_2 n} \leq 2^{\frac{1}{c_1}|I|}.$$
- Poiché $2^{\log_2 n} = n$, otteniamo
$$2^{\frac{1}{c_2}|I|} \leq n \leq 2^{\frac{1}{c_1}|I|}.$$
- Inoltre, $\sqrt{n} = n^{1/2}$. Estraendo la radice quadrata di tutti i termini, il che di nuovo preserva l'ordine perché tutti i termini sono positivi, e usando $(2^x)^{1/2} = 2^{x/2}$, otteniamo
$$2^{\frac{1}{2c_2}|I|} \leq \sqrt{n} \leq 2^{\frac{1}{2c_1}|I|}.$$

Tempo di esecuzione in funzione di $|I|$ (3/4)

- Poiché $S(n) = \Theta(\sqrt{n})$, esistono costanti $a_1, a_2 > 0$ tali che, per ogni n sufficientemente grande,

$$a_1\sqrt{n} \leq S(n) \leq a_2\sqrt{n}.$$

- Usando i limiti su $\sqrt{n}$ in funzione di $|I|$, otteniamo

$$a_1\, 2^{\frac{1}{2c_2}|I|} \leq S(n) \leq a_2\, 2^{\frac{1}{2c_1}|I|}.$$

- Dunque,

$$S(n) = 2^{\Theta(|I|)}.$$

Tempo di esecuzione in funzione di $|I|$ (4/4)

- Il caso peggiore viene raggiunto sui **numeri primi**, e ci sono numeri primi di ogni dimensione
- Per ogni $k \geq 2$ esiste un numero primo con esattamente k cifre binarie, cioè compreso tra 2^{k-1} e 2^k (postulato di Bertrand)
- Quindi, per ogni dimensione dell'input, il massimo numero di passi è raggiunto da un primo n e soddisfa i limiti precedenti

PrimalityTest: tempo di esecuzione

Il tempo di esecuzione nel caso peggiore
dell'algoritmo `PrimalityTest`,
in funzione della dimensione dell'input $|I|$, è

$$T(|I|) = 2^{\Theta(|I|)}.$$

Di conseguenza, l'ordine di crescita del tempo di esecuzione
nel caso peggiore dell'algoritmo `PrimalityTest`
è **esponenziale** nella dimensione dell'input.

Riepilogo

Numero di passi:

- Caso peggiore: n primo, il ciclo viene eseguito fino in fondo
- $S(n) = 3\lfloor\sqrt{n}\rfloor + 1 = \Theta(\sqrt{n})$; un n composto con più piccolo divisore p richiede $3p - 1$ passi

Tempo di esecuzione:

- $|I| = \Theta(\log_2 n)$, quindi $\sqrt{n} = 2^{\Theta(|I|)}$
- $T(|I|) = 2^{\Theta(|I|)}$: **esponenziale** nella dimensione dell'input

Riepilogo

Introduzione: Riepilogo

- Confrontiamo in una tabella le quattro analisi del capitolo
- Raccogliamo i punti chiave del metodo

I quattro algoritmi a confronto

- Per i due algoritmi su array, il tempo di esecuzione è dato sotto l'ipotesi che tutti gli elementi usino un numero costante fissato di bit

algoritmo	parametro n	passi nel caso peggiore $S(n)$	dimensione dell'input $\|I\|$	tempo di esecuzione $T(\|I\|)$
`FindMaximum`	elementi dell'array	$4n$ $= \Theta(n)$	$\Theta(n)$	$\Theta(\|I\|)$
`InsertionSort`	elementi dell'array	$\frac{3n^2+9n-8}{2}$ $= \Theta(n^2)$	$\Theta(n)$	$\Theta(\|I\|^2)$
`ChangeToBase2`	valore del numero	$6\lfloor \log_2 n \rfloor + 11$ $= \Theta(\log_2 n)$	$\Theta(\log_2 n)$	$\Theta(\|I\|)$
`PrimalityTest`	valore del numero	$3\lfloor \sqrt{n} \rfloor + 1$ $= \Theta(\sqrt{n})$	$\Theta(\log_2 n)$	$2^{\Theta(\|I\|)}$

Riepilogo (1/3)

- Il tempo di esecuzione si misura in **passi** (operazioni elementari, ciascuna eseguita in tempo costante); contiamo un passo per ogni esecuzione di una riga di pseudocodice
- Consideriamo il **caso peggiore**: il massimo numero di passi su tutti gli input di una data dimensione
- Quando il numero di esecuzioni di una riga dipende dall'input, una funzione ausiliaria t lo descrive, e cerchiamo l'input che massimizza il totale

Riepilogo (2/3)

- L'analisi ha **due passi**: prima il numero di passi $S(n)$ in funzione di un parametro naturale n, poi il tempo di esecuzione $T(|I|)$ in funzione della dimensione dell'input
- Se n è il numero di elementi di un array (con elementi di dimensione fissata), allora $|I| = \Theta(n)$ e l'ordine di crescita non cambia:

$$S(n) = \Theta(n^k) \quad \text{dà} \quad T(|I|) = \Theta(|I|^k)$$

Riepilogo (3/3)

- Se n è il **valore** del numero in input, allora $|I| = \Theta(\log_2 n)$, cioè
$$n = 2^{\Theta(|I|)}$$
- Un numero di passi $\Theta(\log_2 n)$ è **lineare** in $|I|$
- Un numero di passi $\Theta(\sqrt{n})$ o $\Theta(n)$ è **esponenziale** in $|I|$